

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	ITA05	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	Malli Chandana	Reg. No.:	192472250
Section / Batch:	CSE-AI/2024-2028	Date:	25-08-2026

Code of Conduct

I Malli Chandana(192472250)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Malli Chandana

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

Feature Matching Pipeline

A system must:

A. Read two images

B. Extract features

C. Match features

D. Display matches

Constraints:

- **Robust to scale and rotation**
- **Efficient matching**

Write pseudocode with modular steps for detection, matching, and visualization.

1. Main Pipeline

```
START
↓
Read Image 1 and Image 2
↓
Check Images Loaded Successfully
↓
Convert Images to Grayscale
↓
Detect Keypoints using SIFT
↓
Extract Feature Descriptors
↓
Match Descriptors
↓
Apply Ratio Test
↓
Select Good Matches
↓
Draw Matching Keypoints
↓
Display Matching Result
↓
END
```

2. Pseudocode

Main Program

```
BEGIN

    READ Image1
    READ Image2

    IF Image1 or Image2 is not available THEN
        DISPLAY "Error: Image not found"
        STOP
    END IF

    CONVERT Image1 to grayscale
    CONVERT Image2 to grayscale

    INITIALIZE SIFT detector

    CALL FeatureDetection(Image1, Image2)

    CALL FeatureMatching(Descriptors1, Descriptors2)

    CALL VisualizeMatches(
        Image1,
        Keypoints1,
        Image2,
        Keypoints2,
        GoodMatches
    )

    DISPLAY matching result

END
```

3. Module 1 — Image Input

This module reads the two input images.

```
MODULE ReadImages

    Image1 ← READ first image
    Image2 ← READ second image

    IF Image1 == NULL THEN
        PRINT "First image cannot be loaded"
        RETURN ERROR
    END IF

    IF Image2 == NULL THEN
        PRINT "Second image cannot be loaded"
        RETURN ERROR
    END IF

    RETURN Image1, Image2

END MODULE
```

Purpose

The system first checks whether both images have been successfully loaded. This prevents errors during feature extraction.

4. Module 2 — Preprocessing

```
MODULE Preprocess(Image1, Image2)

    Gray1 ← Convert Image1 to Grayscale
    Gray2 ← Convert Image2 to Grayscale

    RETURN Gray1, Gray2

END MODULE
```

Grayscale conversion reduces the amount of data that needs to be processed and is sufficient for SIFT feature extraction.

5. Module 3 — Feature Detection and Extraction

For the scale and rotation constraints, **SIFT** is a suitable choice because its descriptors are designed to be relatively invariant to changes in image scale and rotation.

```
MODULE FeatureDetection(Gray1, Gray2)

    INITIALIZE SIFT detector

    Keypoints1, Descriptors1 ←
        SIFT.detectAndCompute(Gray1)

    Keypoints2, Descriptors2 ←
        SIFT.detectAndCompute(Gray2)

    IF Descriptors1 or Descriptors2 is NULL THEN
        PRINT "No features detected"
        RETURN ERROR
    END IF

    RETURN Keypoints1, Descriptors1,
        Keypoints2, Descriptors2

END MODULE
```

Output

The module produces:

```
Image 1 → Keypoints1 + Descriptors1
Image 2 → Keypoints2 + Descriptors2
```

A **keypoint** represents an interesting location such as a corner or distinctive texture.

A **descriptor** represents the local visual characteristics around that keypoint.

6. Module 4 — Feature Matching

For efficient matching, **FLANN** can be used to search for similar descriptors.

```
MODULE FeatureMatching(Descriptors1, Descriptors2)

    INITIALIZE FLANN matcher

    Matches ←
        FLANN.knnMatch(
            Descriptors1,
            Descriptors2,
            k = 2
        )

    GoodMatches ← EMPTY LIST

    FOR each pair (Match1, Match2) in Matches DO

        IF Distance(Match1) <
            0.7 × Distance(Match2) THEN

            ADD Match1 to GoodMatches

        END IF

    END FOR

    RETURN GoodMatches

END MODULE
```

7. Ratio Test

The ratio test helps remove unreliable matches.

```
IF

    distance(best match)
        ----- < 0.7
    distance(second best match)

THEN

    ACCEPT match

ELSE

    REJECT match
```

The idea is that the best match should be **significantly better** than the second-best candidate.

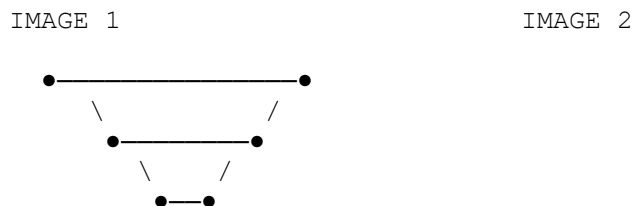
This improves matching accuracy by eliminating ambiguous feature correspondences.

8. Module 5 — Visualization

After obtaining the good matches, the system draws lines between corresponding keypoints.

```
MODULE VisualizeMatches(  
    Image1,  
    Keypoints1,  
    Image2,  
    Keypoints2,  
    GoodMatches  
)  
  
    MatchingImage ← DrawMatches(  
        Image1,  
        Keypoints1,  
        Image2,  
        Keypoints2,  
        GoodMatches  
    )  
  
    DISPLAY MatchingImage  
END MODULE
```

The output will look conceptually like:



Each line represents a pair of corresponding features.

9. Complete Modular Pseudocode

```
BEGIN FEATURE_MATCHING_SYSTEM  
  
    -----  
    MODULE 1: READ IMAGES  
    -----  
  
    Image1 ← Read("image1.jpg")  
    Image2 ← Read("image2.jpg")  
  
    IF Image1 == NULL OR Image2 == NULL THEN
```

```
        PRINT "Error: Images could not be loaded"
        STOP
    END IF
```

```
MODULE 2: PREPROCESSING
```

```
Gray1 ← ConvertToGrayscale(Image1)
Gray2 ← ConvertToGrayscale(Image2)
```

```
MODULE 3: FEATURE DETECTION
```

```
SIFT ← InitializeSIFT()

Keypoints1, Descriptors1 ←
    SIFT.DetectAndCompute(Gray1)

Keypoints2, Descriptors2 ←
    SIFT.DetectAndCompute(Gray2)

IF Descriptors1 == NULL OR
   Descriptors2 == NULL THEN

    PRINT "No sufficient features detected"
    STOP

END IF
```

```
MODULE 4: FEATURE MATCHING
```

```
Matcher ← InitializeFLANN()

CandidateMatches ←
    Matcher.KNNMatch(
        Descriptors1,
        Descriptors2,
        k = 2
    )
```

```
MODULE 5: FILTER MATCHES
```

```
GoodMatches ← EMPTY LIST

FOR each (M1, M2) in CandidateMatches DO

    IF M1.Distance < 0.7 × M2.Distance THEN
        ADD M1 to GoodMatches
    END IF

END FOR
```

MODULE 6: VISUALIZATION

```
Result ← DrawMatches(  
    Image1,  
    Keypoints1,  
    Image2,  
    Keypoints2,  
    GoodMatches  
)  
  
DISPLAY Result  
  
PRINT "Number of good matches:",  
    LENGTH(GoodMatches)
```

```
END FEATURE_MATCHING_SYSTEM
```

10. Handling Scale and Rotation

Scale Robustness

SIFT detects features at multiple scales. Therefore, the same object can still produce corresponding features even when one image contains a **larger or smaller version** of the object.

Small Object



Large Object



The SIFT descriptor is designed to maintain useful correspondence across these scale changes.

Rotation Robustness

SIFT also assigns an orientation to detected keypoints. Therefore, corresponding features can be matched even when the object is rotated.

Image 1

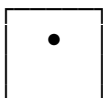


Image 2



This makes SIFT a suitable choice when the assignment specifically requires **scale and rotation robustness**.

11. Efficient Matching Strategy

The major efficiency improvement is to avoid comparing every descriptor with every other descriptor when the number of features becomes large.

FLANN

FLANN = Fast Library for Approximate Nearest Neighbors

It performs approximate nearest-neighbor searches efficiently.

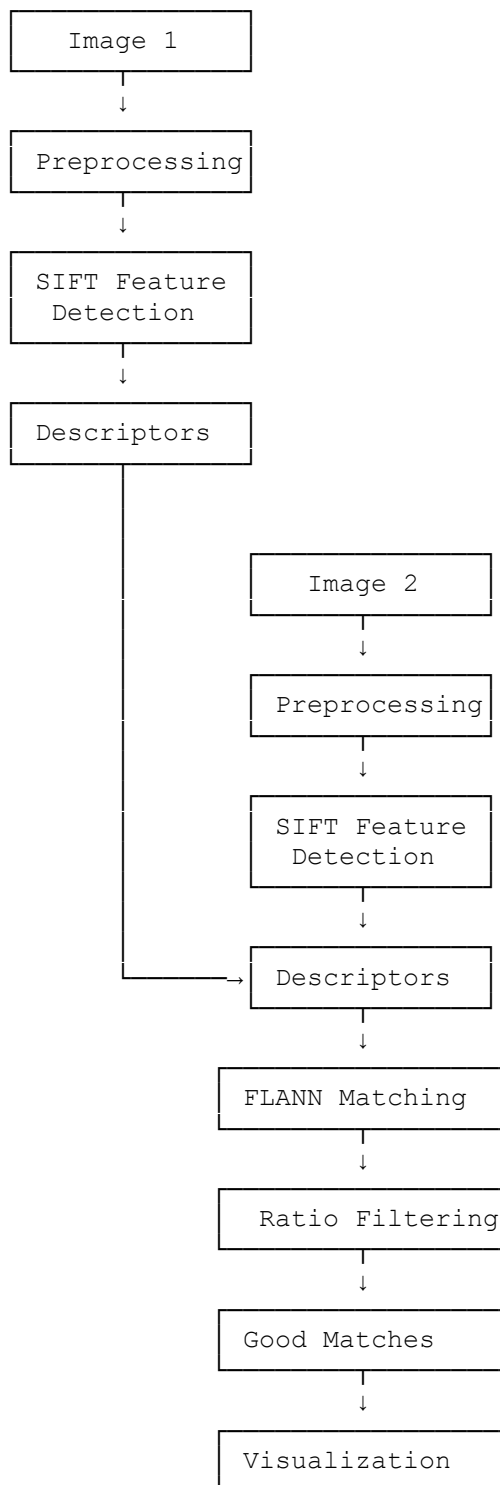
The pipeline becomes:

```
SIFT Features
      ↓
Descriptors
      ↓
FLANN
      ↓
Nearest Neighbors
      ↓
Ratio Test
      ↓
Good Matches
```

Other optimization techniques

1. Convert images to grayscale.
2. Use SIFT descriptors only after detecting useful keypoints.
3. Use FLANN for large descriptor sets.
4. Use the ratio test to eliminate ambiguous matches.
5. Limit the maximum number of features if processing speed is critical.
6. Resize extremely large images when very high resolution is unnecessary.
7. Use RANSAC/homography when geometric verification is required.

12. Final Pipeline



Conclusion

The proposed feature-matching pipeline uses **SIFT for robust feature extraction**, **FLANN for efficient descriptor matching**, and the **ratio test for removing ambiguous matches**. SIFT provides robustness to **scale and rotation**, while FLANN reduces the computational cost of matching large sets of descriptors. Finally, the good matches are visualized by drawing corresponding points and connecting lines between the two images.